Chapter 5

Argumentation among Agents

Iyad Rahwan

1 Introduction

The theory of argumentation is a rich, interdisciplinary area of research spanning philosophy, communication studies, linguistics, and psychology [53]. Argumentation can be seen as a reasoning process consisting of the following four steps:

1. Constructing *arguments* (in *favor* of/*against* a "statement") from available information.

2. Determining the different *conflicts* among the arguments.

3. Evaluating the *acceptability* of the different arguments.

4. Concluding, or defining, the *justified conclusions*.

The argumentation metaphor has found a wide range of applications in both theoretical and practical branches of artificial intelligence and computer science [5, 44]. These applications range from specifying semantics for logic programs [12], to natural language text generation [14], to supporting legal reasoning [4], to medical decision-support [16].

In multiagent systems, argumentation has been used both for automating individual agent reasoning, as well as multiagent interaction. In this chapter, I focus on the latter. I refer the reader to other articles for examples of argumentation-based single-agent reasoning, such as belief revision [15] or planning [3, 41]. For

a more comprehensive overview of argumentation in AI, the reader may refer to the recent books on the subject [7, 46]. Due to space limitations, my coverage cannot do justice to the broad literature on argumentation in multiagent systems. Instead, I chose to give some representative approaches to give the reader an overview of the main ideas and insights.

2 What Is an Argument?

Among argumentation theorists in philosophy, the term "argument" usually refers to "the giving of reasons to support or criticize a claim that is questionable, or open to doubt" [59]. This distinguishes argumentation from deductive mathematical inference, in which the conclusions follow necessarily from the premises. Here, I give a very brief overview of the major approaches to formalizing this notion in the AI literature.

2.1 Arguments as Chained Inference Rules

Formalizations of argumentation using logically constructed structures have their roots in the work of Pollock [34], Lin and Shoham [23], and Vreeswijk [56]. Other approaches include those building on classical logic [6] or logic programming formalisms [8, 49].

A number of recent attempts have been made to provide a general, unifying definition. In this section, I give a very brief overview of Prakken's recent unifying framework [36], since it is quite general and highlights most important concepts. Prakken defines an *argumentation system* as a tuple $(\mathcal{L}, -, \mathcal{R}_s, \mathcal{R}_d, \leq)$, consisting of a logical language $\mathcal{L}$, two disjoint sets of strict rules $\mathcal{R}_s$ and defeasible rules $\mathcal{R}_d$, and a partial order $\leq$ over $\mathcal{R}_d$. The *contrariness function* $- : \mathcal{L} \to 2^{\mathcal{L}}$ captures conflict between formulas, with classical negation $\neg$ being captured by $\neg\varphi \in \overline{\varphi}$ and $\varphi \in \overline{\neg\varphi}$.

A particular knowledge base is a pair $(\mathcal{K}, \leq')$, with $\mathcal{K} \subseteq \mathcal{L}$ divided into the following disjoint sets: $\mathcal{K}_n$ are the necessary *axioms* (cannot be attacked); $\mathcal{K}_p$ are the *ordinary premises*; $\mathcal{K}_a$ are the *assumptions*; $\mathcal{K}_i$ are the *issues*. Finally, $\leq'$ is a partial order on $\mathcal{K} \backslash \mathcal{K}_n$.

From a knowledge base, arguments are built by applying *inference rules* to subsets of $\mathcal{K}$. The left-hand side of the rule $\varphi_1, \ldots, \varphi_n$ is called the *premises* (or *antecedents*), while the right-hand side φ is called the *conclusion* (or *consequent*). A *strict* rule of the form $\varphi_1, \ldots, \varphi_n \to \varphi$ stands for classical implication, while a *defeasible* rule of the form $\varphi_1, \ldots, \varphi_n \Rightarrow \varphi$ means that φ follows *presumably* from the premises. Functions $Perm(A)$, $Conc(A)$ and $Sub(A)$ return premises,

conclusion, and *subarguments* of argument A, respectively. Omitting some details (see [36]), an argument is intuitively any of the following structures:

- $\varphi \in \mathcal{K}$, where $Prem(A) = \{\varphi\}$, $Conc(A) = \varphi$, and $Sub(A) = \{\varphi\}$.
- $A_1, \ldots, A_n \rightarrow \psi$, where $A_1, \ldots, A_n$ are arguments, and there exists in $\mathcal{R}_s$ a strict rule $Conc(A_1), \ldots, Conc(A_n) \rightarrow \psi$.
- $A_1, \ldots, A_n \Rightarrow \psi$, where $A_1, \ldots, A_n$ are arguments, and there exists in $\mathcal{R}_d$ a defeasible rule $Conc(A_1), \ldots, Conc(A_n) \rightarrow \psi$.

In the second and third cases, we define $Prem(A) = Prem(A_1) \cup \cdots \cup Prem(A_n)$, that is, the premises of an argument are the union of the premises of its constituents. Likewise, $Sub(A) = Sub(A_1) \cup \cdots \cup Sub(A_n) \cup \{A\}$. The following example is illustrative:

Example 5.1 (from [36]) *Consider a knowledge base in an argumentation system with* $\mathcal{R}_s = \{p, q \rightarrow s; u, v \rightarrow w\}$, $\mathcal{R}_d = \{p \Rightarrow t; s, r, t \Rightarrow v\}$, $\mathcal{K}_n = \{q\}$, $\mathcal{K}_p = \{p, u\}$, $\mathcal{K}_a = \{r\}$. *We can construct the following arguments:*

$A_1 : p$ $\quad$ $A_5 : A_1 \Rightarrow t$
$A_2 : q$ $\quad$ $A_6 : A_1, A_2 \rightarrow s$
$A_3 : r$ $\quad$ $A_7 : A_3, A_5, A_6 \Rightarrow v$
$A_4 : u$ $\quad$ $A_8 : A_4, A_7 \rightarrow w$

with $Prem(A_8) = \{p, q, r, u\}$, $Conc(A_8) = \{w\}$, *and*
$Sub(A_8) = \{A_1, A_2, A_3, A_4, A_5, A_6, A_6, A_8\}$.

The example above shows how an argument can form part of (the *support* of) another argument. The other possibility is when an argument *attacks* another argument. Various notions of attack have been explored in the literature. For example, an argument can *undercut* another argument by showing that a defeasible rule cannot be applied.

Example 5.2 *In Example 5.1, argument* A_8 *can be undercut by an argument with conclusion* $\overline{A_5}$, *since argument* A_5 *is constructed with a defeasible rule.*

Another way to attack an argument is to *rebut* with an argument that has the opposite conclusion.

Example 5.3 *In Example 5.1, argument* A_8 *can be rebutted on* A_5 *with an argument with conclusion* $\bar{t}$, *or rebutted on* A_7 *with an argument with conclusion* $\bar{v}$.

Rebutters and undercutters were first formalized by Pollock [34]. Yet another way to attack an argument is to *undermine* it by attacking one of its premises (also called *premise attack*), introduced by Vreeswijk [56].

Example 5.4 *In Example 5.1, argument A_8 can be undermined by an argument with conclusion $\overline{p}$, $\overline{r}$, or $\overline{u}$.*

Finally, we say that an argument *A defeats* argument *B* if the former attacks the latter, and is also preferred to it according to some preference relation $\prec$, which is itself based on the nature of the attack as well as the preference relations $\leq$ and $\leq'$ over the formulas involved. Details are beyond this introductory chapter, and can be found elsewhere [36].

2.2 Argument as an Instance of a Scheme

Argumentation schemes are forms (or categories) of argument, representing stereotypical ways of drawing inferences from particular patterns of premises to conclusions in a particular domain (e.g., reasoning about action). Prior to the development of formal models of argumentation, a number of such informal schemes have been proposed [33, 51]. One of the most comprehensive classifications was presented by Walton [58]. For each category, Walton specifies the scheme informally by listing the general form of the premises and conclusion, together with a set of critical questions that can be used to scrutinize the argument by questioning explicit or implicit premises. For example, Walton's "sufficient condition scheme for practical reasoning" may be described as follows [1]:

> In the current circumstances *R*
> We should perform action *A*
> Which will result in new circumstances *S*
> Which will realize goal *G*
> Which will promote some value *V*.

And its associated critical questions include:

> **CQ1:** Are the believed circumstances true?
> **CQ2:** Does the action have the stated consequences?
> **CQ3:** Assuming the circumstances and that the action has the stated consequences, will the action bring about the desired goal?
> **CQ4:** Does the goal realize the value stated?
> **CQ5:** Are there alternative ways of realizing the same consequences?

A number of formalizations of Walton's schemes have been attempted. For example, Atkinson and Bench-Capon [1] formalize the above scheme using Action-based Alternating Transition Systems (AATS), which are a variant of alternating time logic [52]. Walton's schemes have also been influential in the proposed Argument Interchange Format (AIF), which is a community-based effort to produce

an extensible ontology for describing argument structures [11]. Yet another formalization of Walton's scheme was presented by Gordon et al. in their Carneades model [18].

2.3 Abstract Arguments

In the preceding sections, each argument had an explicit internal structure, and relationships between arguments were defined carefully, using various notions of attack they pose or critical questions they raise. In a seminal article, Dung showed that many fundamental aspects of argumentation can be studied without paying attention to the internal structure of the arguments themselves [12]. Thus, an argument can be seen as a node in an argument graph, in which directed arcs capture defeat between arguments. Despite its simplicity, this model is surprisingly powerful.

Definition 5.1 (Argumentation framework) *An* argumentation framework *is a pair* $AF = \langle \mathcal{A}, \rightharpoonup \rangle$ *where* $\mathcal{A}$ *is a finite set of arguments and* $\rightharpoonup \subseteq \mathcal{A} \times \mathcal{A}$ *is a defeat relation. We say that an argument* α defeats *an argument* β *if* $(\alpha, \beta) \in \rightharpoonup$ *(sometimes written* $\alpha \rightharpoonup \beta$*).*

An argumentation framework can be represented as a directed graph in which vertices are arguments and directed arcs characterize defeat among arguments. An example of an argument graph is shown in Figure 5.1. Argument α_1 has two defeaters (i.e., counterarguments) α_2 and α_4, which are themselves defeated by arguments α_3 and α_5, respectively.

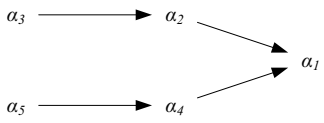


Figure 5.1: A simple argument graph.

3 Evaluating an Argument

To evaluate whether an argument is acceptable (according to some logical semantics), we need to take into account how it interacts with other arguments. This turns out to be a non-trivial problem, and the source of much research [2] and controversy [9]. Let $S^+ = \{\beta \in \mathcal{A} \mid \alpha \rightharpoonup \beta \text{ for some } \alpha \in S\}$. Also let

$\alpha^- = \{\beta \in \mathcal{A} \mid \beta \rightharpoonup \alpha\}$. We first characterize the fundamental notions of conflict-free and defense.

Definition 5.2 (Conflict-free, defense) *Let $\langle \mathcal{A}, \rightharpoonup \rangle$ be an argumentation framework and let $S \subseteq \mathcal{A}$ and let $\alpha \in \mathcal{A}$.*

- *S is* conflict-free *if $S \cap S^+ = \emptyset$.*
- *S* defends *argument α if $\alpha^- \subseteq S^+$. We also say that argument α is* acceptable with respect to *S.*

Intuitively, a set of arguments is *conflict-free* if no argument in that set defeats another. A set of arguments *defends* a given argument if it defeats all its defeaters. In Figure 5.1, for example, $\{\alpha_3, \alpha_5\}$ defends α_1. We now look at different semantics that characterize the *collective acceptability* of a set of arguments.

Definition 5.3 (characteriztic function) *Let $AF = \langle \mathcal{A}, \rightharpoonup \rangle$ be an argumentation framework. The* characteriztic function *of AF is $\mathcal{F}_{AF}$: $2^{\mathcal{A}} \rightarrow 2^{\mathcal{A}}$ such that, given $S \subseteq \mathcal{A}$, we have $\mathcal{F}_{AF}(S) = \{\alpha \in \mathcal{A} \mid S$ defends $\alpha\}$.*

When there is no ambiguity, we use $\mathcal{F}$ instead of $\mathcal{F}_{AF}$. The characteriztic function allows us to capture the collective criteria of admissibility.

Definition 5.4 (Admissible set) *Let S be a conflict-free set of arguments in framework $\langle \mathcal{A}, \rightharpoonup \rangle$. S is* admissible *if it is conflict-free and defends every element in S (i.e., if $S \subseteq \mathcal{F}(S)$).*

Intuitively, a set of arguments is *admissible* if it is a conflict-free set that defends itself against any defeater – in other words, if it is a conflict-free set in which each argument is acceptable with respect to the set itself.

Example 5.5 *In Figure 5.1, the sets $\emptyset$, $\{\alpha_3\}$, $\{\alpha_5\}$, and $\{\alpha_3, \alpha_5\}$ are all admissible simply because they do not have any defeaters. The set $\{\alpha_1, \alpha_3, \alpha_5\}$ is also admissible since it defends itself against both defeaters α_2 and α_4.*

An admissible set *S* is a *complete extension* if and only if *all* arguments defended by *S* are also in *S* (that is, if *S* is a fixed point of the operator $\mathcal{F}$).

Definition 5.5 (Complete extensions) *Let S be a conflict-free set of arguments in framework $\langle \mathcal{A}, \rightharpoonup \rangle$. S is a* complete extension *if $S = \mathcal{F}(S)$.*

Example 5.6 *In Figure 5.1, the admissible set $\{\alpha_3, \alpha_5\}$ is not a complete extension, since it defends α_1 but does not include α_1. Similarly, sets $\{\alpha_3\}$ and $\{\alpha_5\}$ are not complete extensions, since $\mathcal{F}(\{\alpha_3\}) = \{\alpha_3, \alpha_5\}$ and $\mathcal{F}(\{\alpha_5\}) = \{\alpha_3, \alpha_5\}$. The admissible set $\{\alpha_1, \alpha_3, \alpha_5\}$ is the only complete extension, since $\mathcal{F}(\{\alpha_1, \alpha_3, \alpha_5\}) = \{\alpha_1, \alpha_3, \alpha_5\}$.*

Before moving to further refinements of the complete extension, I discuss an equivalent way to characterize complete extensions using *argument labeling* [10]. A labeling specifies which arguments are accepted (labeled `in`), which ones are rejected (labeled `out`), and which ones whose acceptance or rejection could not be decided (labeled `undec`). Labelings must satisfy the condition that an argument is `in` if and only if all of its defeaters are `out`. An argument is `out` if and only if at least one of its defeaters is `in`. Otherwise, it is undecided.

Definition 5.6 (Argument labeling) *Let* $AF = \langle \mathcal{A}, \rightharpoonup \rangle$. *A* labeling *is a total function* $L : \mathcal{A} \rightarrow \{\texttt{in}, \texttt{out}, \texttt{undec}\}$ *such that:*

- $\forall \alpha \in \mathcal{A} \ : \ (L(\alpha) = \texttt{out} \equiv \exists \beta \in \mathcal{A}$ *such that* $(\beta \rightharpoonup \alpha$ *and* $L(\beta) = \texttt{in}))$
- $\forall \alpha \in \mathcal{A} \ : \ (L(\alpha) = \texttt{in} \equiv \forall \beta \in \mathcal{A} \ : \ (\ \textit{if}\ \beta \rightharpoonup \alpha\ \textit{then}\ L(\beta) = \texttt{out}))$

Otherwise, $L(\alpha) = \texttt{undec}$ *(since L is a total function).*

As it turns out, for any labeling satisfying the conditions above, those arguments that happen to be labeled `in` form a complete extension as per Definition 5.5.

If the argument graph contains cycles, there may be more than one complete extension (i.e., more than one legal labeling), each corresponding to a particular consistent and self-defending viewpoint. Consider the following.

Example 5.7 *Consider the graph in Figure 5.2. Here, we have three complete extensions:* $\{\alpha_3\}$, $\{\alpha_1, \alpha_3\}$, *and* $\{\alpha_2, \alpha_3\}$.

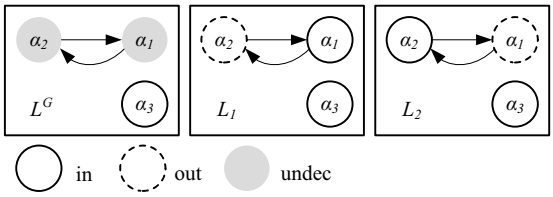


Figure 5.2: Graph with three labelings/complete extensions.

Now, we turn to further refinements of the notion of complete extension.

Definition 5.7 (Refinements of the complete extension) *Let S be a conflict-free set of arguments in framework* $\langle \mathcal{A}, \rightharpoonup \rangle$.

- *S is a* grounded extension *if it is the minimal (w.r.t. set-inclusion) complete extension (or, alternatively, if S is the least fixed-point of* $\mathcal{F}(.)$*).*

- *S is a* preferred extension *if it is a maximal (w.r.t. set-inclusion) complete extension (or, alternatively, if S is a maximal admissible set).*
- *S is a* stable extension *if* $S^+ = \mathcal{A} \backslash S$.
- *S is a* semi-stable extension *if S is a complete extension of which* $S \cup S^+$ *is maximal.*

A *grounded extension* contains all the arguments which are not defeated, as well as the arguments which are defended directly or indirectly by non-defeated arguments. This can be seen as a non-committal view (characterized by the *least* fixed point of $\mathcal{F}$). As such, there always exists a unique grounded extension. Dung [12] showed that in finite argumentation systems, the grounded extension can be obtained by an iterative application of the characteriztic function $\mathcal{F}$ to the empty set. For example, in Figure 5.1 the grounded extension (and the only complete extension) can be obtained as follows:

- $\mathcal{F}^1(\emptyset) = \{\alpha_3, \alpha_5\}$;
- $\mathcal{F}^2(\emptyset) = \mathcal{F}(\mathcal{F}^1(\emptyset)) = \{\alpha_1, \alpha_3, \alpha_5\}$;
- $\mathcal{F}^3(\emptyset) = \mathcal{F}(\mathcal{F}^2(\emptyset)) = \{\alpha_1, \alpha_3, \alpha_5\} = \mathcal{F}^2(\emptyset)$.

Similarly, in Figure 5.2, the grounded extension is $\{\alpha_3\}$, which is the minimal complete extension w.r.t. set inclusion.

A *preferred extension* is a bolder, more committed position that cannot be extended – by accepting more arguments – without causing inconsistency. Thus a preferred extension can be thought of as a maximal consistent set of hypotheses. There may be multiple preferred extensions, and the grounded extension is included in all of them.

Example 5.8 *In Figure 5.1,* $\{\alpha_1, \alpha_3, \alpha_5\}$ *is the only preferred extension. But in Figure 5.2, there are two preferred extensions:* $\{\alpha_1, \alpha_3\}$ *and* $\{\alpha_2, \alpha_3\}$*, which are the maximal complete extension w.r.t. set inclusion.*

Finally, a set of arguments is a *stable extension* if it is a preferred extension that defeats every argument that does not belong to it. As expected, stable extensions may now always exist. An alternative definition is a *semi-stable extension*, which satisfies the weaker condition that the set of arguments defeated is maximal.

Caminada [10] established a correspondence between properties of labelings and the different extensions. Let $AF = \langle \mathcal{A}, \rightharpoonup \rangle$ be an argumentation framework, and L a labeling over AF. Define $\texttt{in}(L) = \{\alpha \in \mathcal{A} \mid L(\alpha) = \texttt{in}\}$; $\texttt{out}(L) = \{\alpha \in \mathcal{A} \mid L(\alpha) = \texttt{out}\}$; and $\texttt{undec}(L) = \{\alpha \in \mathcal{A} \mid L(\alpha) = \texttt{undec}\}$. These are summarized in Table 5.1.

Extensions	Restrictions on labelings
complete	all labelings
grounded	minimal `in`, or equivalently minimal `out`, or equivalently maximal `undec`
preferred	maximal `in`, or equivalently maximal `out`
semi-stable	minimal `undec`
stable	empty `undec`

Table 5.1: The relationships between extensions and labelings.

Now that the acceptability of sets of arguments is defined, we can define the status of any individual argument.

Definition 5.8 (Argument status) *Let $\langle \mathcal{A}, \rightharpoonup \rangle$ be an argumentation system, and $\mathcal{E}_1, \ldots, \mathcal{E}_n$ its extensions under a given semantics. Let $\alpha \in \mathcal{A}$.*

1. *α is* skeptically accepted *iff $\alpha \in \mathcal{E}_i$, $\forall \mathcal{E}_i$ with $i = 1, \ldots, n$.*

2. *α is* credulously accepted *iff $\exists \mathcal{E}_i$ such that $\alpha \in \mathcal{E}_i$.*

3. *α is* rejected *iff $\nexists \mathcal{E}_i$ such that $\alpha \in \mathcal{E}_i$.*

An argument is *skeptically accepted* if it belongs to all extensions under the adopted semantics. Intuitively, an argument is skeptically accepted if it can be accepted without making any hypotheses beyond what is surely self-defending. On the other hand, an argument is *credulously accepted* on the basis that it belongs to at least one extension. Intuitively, an argument is credulously accepted if there is a possible consistent set of hypotheses in which it is consistent. If an argument is neither skeptically nor credulously accepted, there is no basis for accepting it, and it is therefore *rejected*.

4 Argumentation Protocols

So far, I outlined some methods for evaluating an argument given an existing collection of arguments. In a multiagent system, however, the arguments are not all available a priori. Instead, they are presented by the agents during their argumentative dialogue. This raises the question of how such argumentation dialogues are to be regulated. For example, one should not be able to make statements that are completely irrelevant, or to contradict oneself. An argumentation protocol is, therefore, a set of rules that govern the argumentation process.

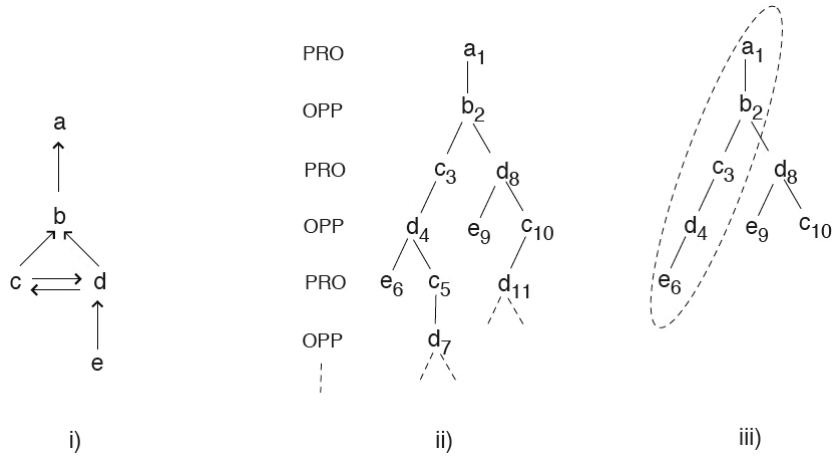


Figure 5.3: Argumentation framework and dispute tree. (i) shows an argumentation framework, (ii) shows the dispute tree induced in a, and (iii) shows the dispute tree induced by a under protocol G, with the winning strategy encircled.

4.1 Abstract Argument Games

Rules for governing argumentation can be specified without regard to the specific internal structure of the arguments. These protocols typically assume the presence of two agents, one PRO (the proponent) and the other OPP (the opponent). Dialogue begins with PRO asserting an argument x. Then PRO and OPP take turns, in a sequence of moves called a *dispute*, where each player makes an argument that attacks its counterpart's last move. A player wins a dispute if its counterpart cannot make a counterattack. But the counterpart may try a different line of attack, creating a new dispute. This results in a *dispute tree* structure that represents the dialogue.

The idea is that there is an implicit abstract argument graph, distributed in the agents' heads, so to speak. By following a specific protocol, the two agents arrive at an outcome (about the status of a particular argument) that corresponds to a particular semantics over the implicit graph.

Example 5.9 *Consider two agents arguing the argument graph shown in Figure 5.3, taken from [30]. In Figure 5.3(i), we see the underlying (usually implicit) argument graph. Figure 5.3(ii) shows the corresponding dispute tree in which PRO presents argument a, OPP counters with argument b, PRO then counters once with argument c and once with argument d, and so on. Note that this dispute tree is infinite, since agents are able to repeat counterarguments due to the presence of cycles in the argument graph. As shown in the figure, arguments in the dispute tree can be indexed to capture repetition of the same argument from the graph.*

We now present a definition of *winning strategy* adapted from [30].

Definition 5.9 (Winning strategy) *Given argument graph $\langle \mathcal{A}, \rightharpoonup \rangle$ and a dispute tree T with root a. A subtree T' is a* winning strategy *for a iff:*

1. *The set of disputes $D_{T'}$ in T' is a non-empty and finite set such that each $d \in D_{T'}$ is finite and is won by PRO (terminates in an argument moved by PRO).*

2. *$\forall d \in D_{T'}$, $\forall d'$ such that d' is a subdispute (i.e., sub-sequence with the same head) of d, and the last move in d' is argument x played by PRO, then for any y such that $y \rightharpoonup x$, there is $d'' \in D_{T'}$ such that d' appended with y is a subdispute of d''.*

PRO is guaranteed to win if it plays the moves described in the winning strategy subtree. The second requirement ensures that all possible objections that can possibly be raised by OPP in one dispute can be neutralized successfully by PRO in the same or some other dispute in the subtree.

Note that creating a dispute tree (or subtree) only requires that every argument presented is a defeater of an argument that has already been presented. Adding further restrictions on the moves of different players can be captured by subtrees of the dispute tree. By adding such restrictions carefully, we can generate dialogue outcomes that correspond to well-known semantics. To illustrate the above, consider the following simple protocol:

Definition 5.10 (Protocol G) *Given argument graph $\langle \mathcal{A}, \rightharpoonup \rangle$, and a dispute D whose tail is argument x. Let $PRO(D)$ be the arguments uttered by PRO.*

- *If the dispute length is odd (next move is by OPP), then the possible next moves are $\{y \mid y \rightharpoonup x\}$.*

- *If the dispute length is even (next move is by PRO), then the possible next moves are $\{y \mid y \rightharpoonup x$ and $y \notin PRO(D)\}$.*

By simply prohibiting PRO from repeating himself, we ensure that the dispute tree is finite. A more striking consequence of this simple restriction is that PRO can only win if the argument at the root is in the grounded extension.

Theorem 5.1 ([30]) *Let $\langle \mathcal{A}, \rightharpoonup \rangle$ be an argument graph. There exists a winning strategy T for x under protocol G such that the set of arguments uttered by PRO in T is conflict-free, if and only if x is in the grounded extension of $\langle \mathcal{A}, \rightharpoonup \rangle$.*

Going back to Figure 5.3, note that the grounded extension of the graph in Figure 5.3(i) is $\{a, c, e\}$. Figure 5.3(iii) shows the dispute tree induced by argument a under protocol G, with the winning strategy $\{(a_1 - b_2 - c_3 - d_4 - e_6)\}$.

As it turns out, it is possible to use different protocol restrictions in order to capture different semantics. Suppose we wish to create dialogues in which PRO wins if and only if the argument in question is in at least one preferred extension (i.e., is credulously accepted under preferred semantics, recalling Definition 5.8). This can be achieved by a protocol in which two restrictions apply. OPP is not allowed to repeat its arguments. And PRO is allowed to repeat its arguments but cannot present a self-attacking argument, or an argument that conflicts (attacks or is attacked by) with another argument it already stated in the dispute. For a more comprehensive discussion of abstract argument games, including protocols that implement other semantics, refer to [30].

4.2 Dialogue Systems

Another approach to specifying argumentation protocols is the so-called *dialogue systems* approach (also known as *dialogue games*). This approach was initiated by Australian logician Charles L. Hamblin [19]. Unlike abstract argument games, this approach relies on having the explicit contents of the arguments presented.

As an example, Table 5.2 shows a specification of a persuasion protocol introduced by Prakken [37]. The left column shows the list of *speech acts* that can be used by agents (the notion of speech acts dates back to Searle [48]). An agent can assert a proposition by uttering "*claim* φ", and can retract its own claims by uttering "*retract* φ". An agent can also concede a proposition claimed by its counterpart by uttering "*concede* φ", or challenge such a claim by uttering "*why* φ". Finally, an agent can support its claims, if challenged, by uttering "φ *since S*". With each speech act, Prakken associates two legal replies, one corresponding to surrender (e.g., by conceding to a claim made by the counterpart), and the other corresponding to attack (e.g., by challenging a claim or elements of an argument *S* presented by the counterpart).

This protocol generates a tree structure in which each utterance (i.e., move) is a node, and its possible responses are its children. Termination is defined in terms of the *dialogical status* of each move. A move is *in* if it is surrendered or if all its attacking replies are *out*. Conversely, a move is *out* if it has a reply that is *in*.[1] Whether the proponent or the opponent wins depends on whether the initial move is in or out. It is possible to also impose the requirement that moves must be *relevant*, meaning it would make its speaker the current winner. To illustrate,

[1] Note that assigning dialogical status bears resemblance (but is not identical) to the labeling of abstract argument graphs discussed in Section 3.

Acts	**Intuitive Meaning**	**Attacks**	**Surrenders**
claim φ	Assert φ is true	*why* φ	*concede* φ
φ *since S*	Support φ by argument *S*	*why* ψ(ψ ∈ *S*) φ′ *since S*′ (defeats φ *since S*)	*concede* ψ (ψ ∈ *S*) *concede* φ
why φ	Challenge φ	φ *since S*	*retract* φ
concede φ	Concede φ claimed by other		
retract φ	Take back own claim φ		

Table 5.2: An example L_c in Prakken's framework [35].

suppose agent P has knowledge base $\{p, p \rightarrow_{r_1} q, p \rightarrow_{r_2} r, p \wedge s \rightarrow_{r_3} r_2 > r_4\}$ and agent O has knowledge base $\{t, t \rightarrow_{r_4} \neg r\}$. These knowledge bases are specified in Prakken and Sartor's argument-based, prioritized, extended logic programming language, in which rules are annotated, allowing for rules that support preferences between other rules [38].[2] The following dialogue is consistent with the above protocol [37], with the target of each move indicated between square brackets:

$P_1[-]$: *claim* r — $O_2[P_1]$: *why* r
$P_3[O_2]$: r *since* $q, q \Rightarrow r$ — $O_4[P_3]$: *why* q
$P_5[O_4]$: q *since* $p, p \Rightarrow q$ — $O_6[P_5]$: *concede* $p \Rightarrow q$
$O_7[P_5]$: *why* p

Note that at this point, player P has many possible moves. It can retract its claim or premises of its argument, or give an argument in support of p. It was also possible for player O to make the following against P's original claim:

$O_7[P_3]$: $\neg r$ *since* $t, t \Rightarrow \neg r$

To this, P may respond with a priority argument, supporting the claim that rule r_2 takes precedence over rule r_4, thus showing that P_3 strictly defeats O_7:

$P_8[O_7]$: $r_2 > r_4$ *since* $p, s, p \wedge s \Rightarrow r_2 > r_4$

At this point, P_1 is *in*, but the dialogue may continue, with O conceding or making further challenges, and so on.

Various other dialogue systems have been presented in the literature, such as Walton and Krabbe's PDD [57] and McBurney and Parsons's Agent Dialogue Framework [28]. A longer discussion and comparison of different dialogue systems for persuasion was presented by Prakken [35].

The above approaches are related to the earlier work on so-called *game seman-*

[2]Note that these implications are not classical, thus they do not satisfy contraposition.

tics for logic, which was pioneered by logicians such as Paul Lorenzen [24] and Jaakko Hintikka [20]. Although many specific instantiations of this notion have been presented in the literature, the general idea is as follows. Given some specific logic, the truth value of a formula is determined through a special-purpose, multi-stage dialogue game between two players, the *verifier* and *falsifier*. The formula is considered true precisely when the verifier has a winning strategy, while it will be false whenever the falsifier has the winning strategy. Similar ideas have been used to implement dialectical proof-theories for defeasible reasoning (e.g., by Prakken and Sartor [38]).

It is worth mentioning that, in addition to the generic protocols presented in the last two sections, various domain-specific protocols have appeared in the literature. These protocols are usually more complex, and are linked to argumentation schemes from specific domains, enabling agents to argue about what action to take [1], the risks involved in various decisions [27], or about the properties of items involved in negotiation [29].

5 Strategic Argumentation and Game Theory

In all of the approaches above, we saw a specification of the argumentation protocol, specifying the set of possible moves that can be made by the agents. When it comes to the behavior of the agents who use this protocol, some protocols were extremely prescriptive, giving exactly a single option at a time. Other protocols (e.g., Prakken's persuasion protocol) gave agents some choice of what to do. The behavior of an agent, its so-called *strategy*, significantly influences the outcome of the dialogue (e.g., who wins), as well as its dynamics (e.g., whether it will terminate in a short number of moves).

To address this issue, researchers started exploring strategies for agents to choose their next moves. For example, in their dialogue system, Parsons et al. [32] defined a number of so-called *attitudes* of an agent, which specify the criteria according to which it evaluates and asserts arguments. These attitudes assume the availability of a preference relation over arguments, which can be used to compare their relative strength. In terms of asserting arguments, a *confident* agent can assert any proposition for which it can construct an argument, while a *careful* agent can do so only if it can construct such an argument and cannot construct a stronger counterargument. A *thoughtful* agent, on the other hand, can assert a proposition only if it can construct an acceptable argument for the proposition. Conversely, when it comes to evaluating arguments presented by the opponent, an agent can have one of the following attitudes. A *credulous* agent accepts a proposition if it can construct an argument for it, while a *cautious* agent does so only if it is also unable to construct a stronger counterargument. A *skeptical* agent,

however, accepts an argument only if it can construct an acceptable argument for the proposition.

Another approach to designing heuristics is to use aspects related to social constructs such as rights and obligations [21], or mental states of agents such as their beliefs, desires, and intentions [22]. A more detailed discussion of some of these issues can be found elsewhere [45].

While heuristic approaches to designing argumentation strategies are informative, they typically only address a subset of the possible strategies. A more comprehensive study of strategic argumentation must rely on a more systematic approach, and the appropriate framework for doing so is provided by the theory of games (or *game theory*), which was pioneered by von Neuman and Morgenstern [55]. A setting of strategic interaction is modeled as a *game*, which consists of a set of players, a set of actions available to them, and a rule that determines the outcome given players' chosen actions. In an argumentation scenario, the set of actions are typically the set of argumentative moves (e.g., asserting a claim or challenging a claim), and the outcome rule is the criterion by which arguments are evaluated (e.g., the judge's attitude). Generally, game theory can be used to achieve two goals:

1. undertake precise analysis of interaction in particular strategic settings, with a view to predicting the outcome;

2. design rules of the game in such a way that self-interested agents behave in some desirable way (e.g., tell the truth); this is called *mechanism design*.

Both these approaches are quite useful for the study of argumentation in multi-agent systems. On the one hand, an agent may use game theory to analyze a given argumentative situation in order to choose the best strategy. On the other hand, we may use mechanism design to design the rules (e.g., argumentation protocol) in such a way as to promote good argumentative behavior.

5.1 Glazer and Rubinstein's Model

One of the earliest attempts at game-theoretic analysis of argumentation was presented by microeconomists Glazer and Rubinstein [17]. The authors explore the mechanism design problem of constructing rules of debate that maximize the probability that a listener reaches the right conclusion given arguments presented by two debaters. They study a very restricted setting, in which the world state is described by a vector $\omega = (w_1, \ldots, w_5)$, where each "aspect" w_i has two possible values: 1 and 2. If $w_i = j$ for $j \in \{1,2\}$, we say that aspect w_i supports outcome O_j. Presenting an argument amounts to revealing the value of some w_i. The setting is modeled as an extensive-form game and analyzed. In particular,

the authors investigate various combinations of *procedural rules* (stating in which order and what sorts of arguments each debater is allowed to state) and *persuasion rules* (stating how the outcome is chosen by the listener). In terms of procedural rules, the authors explore: (1) *one-speaker debate* in which one debater chooses two arguments to reveal; (2) *simultaneous debate* in which the two debaters simultaneously reveal one argument each; and (3) *sequential debate* in which one debater reveals one argument followed by one argument by the other. Glazer and Rubinstein investigate a variety of persuasion rules. For example, in one-speaker debate, one rule analyzed by the authors states that "a speaker wins if and only if he presents two arguments from $\{a_1, a_2, a_3\}$ or $\{a_4, a_5\}$." In a sequential debate, one persuasion rule states that "if debater D_1 argues for aspect a_3, then debater D_2 wins if and only if he counter-argues with aspect a_4."

Note that these kinds of rules are arbitrary and do not follow an intuitive notion of persuasion (e.g., like skepticism). Hence, there is no concept of how the logical structure of the information presented by different players constrains the decision made by the judge. In the remainder of this section, I present a game-theoretic account of strategic argumentation based on abstract argument graphs.

5.2 Game Theory Background

This section gives a brief background on key game-theoretic concepts [25]. Readers already familiar with game theory may skip this section. The field of game theory studies strategic interactions of self-interested agents. We assume that there is a set of self-interested agents, denoted by I. We let $\theta_i \in \Theta_i$ denote the *type* of agent i, which is drawn from some set of possible types Θ_i. The type represents the private information and preferences of the agent. An agent's preferences are over *outcomes* $o \in \mathcal{O}$, where $\mathcal{O}$ is the set of all possible outcomes. We assume that an agent's preferences can be expressed by a utility function $u_i(o, \theta_i)$, which depends on both the outcome, o, and the agent's type, θ_i. Agent i prefers outcome o_1 to o_2 when $u_i(o_1, \theta_i) > u_i(o_2, \theta_i)$.

When agents interact, we say that they are playing *strategies*. A strategy for agent i, $s_i(\theta_i)$, is a plan that describes what actions the agent will take for every decision that the agent might be called upon to make, for each possible piece of information that the agent may have at each time it is called to act. That is, a strategy can be thought of as a complete contingency plan for an agent. We let Σ_i denote the set of all possible strategies for agent i, and thus $s_i(\theta_i) \in \Sigma_i$. When it is clear from the context, we will drop the θ_i in order to simplify the notation. We let *strategy profile* $s = (s_1(\theta_1), \ldots, s_I(\theta_I))$ denote the outcome that results when each agent i is playing strategy $s_i(\theta_i)$. As a notational convenience we define

$$s_{-i}(\theta_{-i}) = (s_1(\theta_i), \ldots, s_{i-1}(\theta_{i-1}), s_{i+1}(\theta_{i+1}), \ldots, s_I(\theta_I))$$

and thus $s = (s_i, s_{-i})$. We then interpret $u_i((s_i, s_{-i}), \theta_i)$ to be the utility of agent i with type θ_i when all agents play strategies specified by strategy profile $(s_i(\theta_i), s_{-i}(\theta_{-i}))$. Similarly, we also define:

$$\theta_{-i} = (\theta_1, \ldots, \theta_{i-1}, \theta_{i+1}, \ldots, \theta_I)$$

Since the agents are all self-interested, they will try to choose strategies that maximize their own utility. Since the strategies of other agents also play a role in determining the outcome, the agents must take this into account. The *solution concepts* in game theory determine the outcomes that will arise if all agents are rational and strategic. The most well-known solution concept is the *Nash equilibrium*. A Nash equilibrium is a strategy profile in which each agent is following a strategy that maximizes its own utility, given its type and the strategies of the other agents.

Definition 5.11 (Nash equilibrium) *A strategy profile* $s^* = (s_1^*, \ldots, s_I^*)$ *is a* Nash equilibrium *if no agent has an incentive to change its strategy, given that no other agent changes. Formally,* $\forall i, \forall s_i', u_i(s_i^*, s_{-i}^*, \theta_i) \geq u_i(s_i', s_{-i}^*, \theta_i)$

Although the Nash equilibrium is a fundamental concept in game theory, it does have several weaknesses. First, there may be multiple Nash equilibria and so agents may be uncertain as to which equilibrium they should play. Second, the Nash equilibrium implicitly assumes that agents have perfect information about all other agents, including the other agents' preferences.

A stronger solution concept in game theory is the *dominant-strategy equilibrium*. A strategy s_i is said to be *dominant* if by playing it, the utility of agent i is maximized no matter what strategies the other agents play.

Definition 5.12 (Dominant strategy) *A strategy* s_i^* *is* dominant *if*

$$\forall s_{-i},\ \forall s_i',\ u_i(s_i^*, s_{-i}, \theta_i) \geq u_i(s_i', s_{-i}, \theta_i).$$

Sometimes, we will refer to a strategy satisfying the above definition as *weakly dominant*. If the inequality is strict (i.e., $>$ instead of $\geq$), we say that the strategy is *strictly dominant*. A *dominant-strategy equilibrium* is a strategy profile where each agent is playing a dominant strategy. This is a very robust solution concept since it makes no assumptions about what information the agents have available to them, nor does it assume that all agents know that all other agents are being rational (i.e., trying to maximize their own utility). However, there are many strategic settings where no agent has a dominant strategy.

5.2.1 Mechanism Design

The problem that mechanism design studies is how to ensure that a desirable system-wide outcome or decision is made when there is a group of self-interested agents who have preferences over the outcomes. In particular, we often want the outcome to depend on the preferences of the agents. This is captured by a *social choice function*.

Definition 5.13 (Social choice function) *A* social choice function *is a rule* $f : \Theta_1 \times \ldots \times \Theta_I \to \mathcal{O}$*, that selects some outcome* $f(\theta) \in \mathcal{O}$*, given agent types* $\theta = (\theta_1, \ldots, \theta_I)$.

The challenge, however, is that the types of the agents (the $\theta_i's$) are private and known only to the agents themselves. Thus, in order to select an outcome with the social choice function, one has to rely on the agents to reveal their types. However, for a given social choice function, an agent may find that it is better off if it does not reveal its type truthfully, since by lying it may be able to cause the social choice function to choose an outcome that it prefers. Instead of trusting the agents to be truthful, we use a *mechanism* to try to reach the correct outcome.

A mechanism $\mathcal{M} = (\Sigma, g(\cdot))$ defines the set of allowable strategies that agents can chose, with $\Sigma = \Sigma_1 \times \cdots \times \Sigma_I$ where Σ_i is the strategy set for agent i, and an outcome function $g(s)$ that specifies an outcome o for each possible strategy profile $s = (s_1, \ldots, s_I) \in \Sigma$. This defines a game in which agent i is free to select any strategy in Σ_i, and, in particular, will try to select a strategy that will lead to an outcome that maximizes its own utility. We say that a mechanism *implements* social choice function f if the outcome induced by the mechanism is the same outcome that the social choice function would have returned if the true types of the agents were known.

Definition 5.14 (Implementation) *Mechanism* $\mathcal{M} = (\Sigma, g(\cdot))$ implements *social choice function* f *if there exists an equilibrium* s^* *s.t.* $\forall \theta \in \Theta$, $g(s^*(\theta)) = f(\theta)$.

While the definition of a mechanism puts no restrictions on the strategy spaces of the agents, an important class of mechanisms are the *direct-revelation mechanisms* (or simply *direct mechanisms*).

Definition 5.15 (Direct-revelation mechanism) *A* direct-revelation mechanism *is a mechanism in which* $\Sigma_i = \Theta_i$ *for all i, and* $g(\theta) = f(\theta)$ *for all* $\theta \in \Theta$.

In other words, a direct mechanism is one where the strategies of the agents are to announce a type, θ_i', to the mechanism. While it is not necessary that $\theta_i' = \theta_i$, the important *revelation principle* (see below for more details) states that if a social choice function, $f(\cdot)$, can be implemented, then it can be implemented by a direct

mechanism where every agent reveals its true type [25]. In such a situation, we say that the social choice function is *incentive compatible*.

Definition 5.16 (Incentive compatible) *The social choice function $f(\cdot)$ is* incentive compatible *(or* truthfully implementable*) if the direct mechanism $\mathcal{M} = (\Theta, g(\cdot))$ has an equilibrium s^* such that $s_i^*(\theta_i) = \theta_i$.*

If the equilibrium concept is the dominant-strategy equilibrium, then the social choice function is *strategyproof*. In this chapter we will on occasion call a mechanism incentive compatible or strategyproof. This means that the social choice function that the mechanism implements is incentive compatible or strategyproof.

5.2.2 The Revelation Principle

Determining whether a particular social choice function can be implemented, and in particular, finding a mechanism that implements a social choice function appears to be a daunting task. In the definition of a mechanism, the strategy spaces of the agents are unrestricted, leading to an infinitely large space of possible mechanisms. However, the *revelation principle* states that we can limit our search to a special class of mechanisms [25, Ch. 14].

Theorem 5.2 (Revelation principle) *If there exists some mechanism that implements social choice function f in dominant strategies, then there exists a direct mechanism that implements f in dominant strategies and is truthful.*

The intuitive idea behind the revelation principle is fairly straightforward. Suppose that you have a possibly very complex mechanism, $\mathcal{M}$, which implements some social choice function, f. That is, given agent types $\theta = (\theta_1, \ldots, \theta_I)$ there exists an equilibrium $s^*(\theta)$ such that $g(s^*(\theta)) = f(\theta)$. Then, the revelation principle states that it is possible to create a new mechanism, $\mathcal{M}'$, which, when given θ, will then execute $s^*(\theta)$ on behalf of the agents and then select outcome $g(s^*(\theta))$. Thus, each agent is best off revealing θ_i, resulting in $\mathcal{M}'$ being a truthful, direct mechanism for implementing social choice function f.

The revelation principle is a powerful tool when it comes to studying implementation. Instead of searching through the entire space of mechanisms to check whether one implements a particular social choice function, the revelation principle states that we can restrict our search to the class of truthful, direct mechanisms. If we cannot find a mechanism in this space that implements the social choice function of interest, then there does not exist any mechanism that will do so.

5.3 Argumentation Mechanism Design

In this section I define the mechanism design problem for abstract argumentation, leading to so-called *argumentation mechanism design* (ArgMD) (this work was introduced in [39, 42]).

Let $AF = \langle \mathcal{A}, \mathcal{R} \rangle$ be an argumentation framework with a set of arguments $\mathcal{A}$ and a binary defeat relation $\mathcal{R}$. I define a mechanism with respect to AF and semantics $\mathcal{S}$, and I assume that there is a set of I self-interested agents. I define an agent's type to be its set of arguments.

Definition 5.17 (Agent type) *Given an argumentation framework $\langle \mathcal{A}, \mathcal{R} \rangle$, the* type *of agent i, $\mathcal{A}_i \subseteq \mathcal{A}$, is the set of arguments that the agent is capable of putting forward.*

There are two things to note about this definition. First, an agent's type can be seen as a reflection of its expertise or domain knowledge. For example, medical experts may only be able to comment on certain aspects of forensics in a legal case, whereas a defendant's family and friends may be able to comment on its character. Also, such expertise may overlap, so agent types are not necessarily disjoint. For example, two medical doctors might have some identical argument.

The second thing to note about the definition is that agent types do not include the defeat relation. In other words, I implicitly assume that the notion of defeat is common to all agents. That is, given two arguments, no agent would dispute whether one attacks another. This is a reasonable assumption in systems where agents use the same logic to express arguments or at least multiple logics for which the notion of defeat is accepted by everyone (e.g., conflict between a proposition and its negation). Disagreement over the defeat relation itself requires a form of hierarchical (meta) argumentation, which is a powerful concept, but can be reduced to a standard argument graph [31].

Given the agents' types (argument sets), a social choice function f maps a type profile into a subset of arguments: $f : 2^{\mathcal{A}} \times \ldots \times 2^{\mathcal{A}} \rightarrow 2^{\mathcal{A}}$. Denote by $Acc(\langle \mathcal{A}, \mathcal{R} \rangle, \mathcal{S}) \subseteq \mathcal{A}$ the set of acceptable arguments according to semantics $\mathcal{S}$.[3] It is possible to define *argument acceptability* social choice functions.

Definition 5.18 (Argument acceptability social choice functions) *Given an argumentation framework $\langle \mathcal{A}, \mathcal{R} \rangle$ with semantics $\mathcal{S}$, and given an agent type profile $(\mathcal{A}_1, \ldots, \mathcal{A}_I)$, the* argument acceptability social choice function *f is defined as the set of acceptable arguments given the semantics $\mathcal{S}$. That is,*

$$f(\mathcal{A}_1, \ldots, \mathcal{A}_I) = Acc(\langle \mathcal{A}_1 \cup \ldots \cup \mathcal{A}_I, \mathcal{R} \rangle, \mathcal{S}).$$

[3] Here, assume that $\mathcal{S}$ specifies both the classical semantics used (e.g., grounded, preferred, stable) as well as the acceptance attitude used (e.g., skeptical or credulous).

As is standard in the mechanism design literature, assume that agents have preferences over the outcomes $o \in 2^{\mathcal{A}}$, and we represent these preferences using utility functions where $u_i(o, \mathcal{A}_i)$ denotes agent i's utility for outcome o when its type is argument set $\mathcal{A}_i$.

Agents may not have an incentive to reveal their true type because they may be able to influence the final argument status assignment by lying, and thus obtain higher utility. There are two ways that an agent can lie in our model. On the one hand, an agent might create new arguments that it does not have in its argument set. In the rest of the chapter I will assume that there is an *external verifier* that is capable of checking whether it is possible for a particular agent to actually make a particular argument. Informally, this means that presented arguments, while still possibly defeasible, must at least be based on some sort of demonstrable "plausible evidence." If an agent is caught making up arguments, then it will be removed from the mechanism. For example, in a court of law, any act of perjury by a witness is punished, at the very least, by completely discrediting all evidence produced by the witness. Moreover, in a court of law, arguments presented without any plausible evidence are normally discarded (e.g., "*I did not kill him, since I was abducted by aliens at the time of the crime!*"). For all intents and purposes this assumption (also made by Glazer and Rubinstein [17]) removes the incentive for an agent to make up facts.

A more insidious form of manipulation occurs when an agent decides to *hide* some of its arguments. By refusing to reveal certain arguments, an agent might be able to break defeat chains in the argument framework, thus changing the final set of acceptable arguments. For example, a witness may hide evidence that implicates the defendant if the evidence also undermines the witness's own character. I will focus on this form of lying in this chapter.

As mentioned earlier, a strategy of an agent specifies a complete plan that describes what action the agent takes for every decision that a player might be called upon to make, for every piece of information that the player might have at each time that it is called upon to act. In our model, the actions available to an agent involve announcing sets of arguments. Thus a strategy $s_i \in \Sigma_i$ for agent i would specify for each possible subset of arguments that could define its type, what set of arguments to reveal. For example, a strategy might specify that an agent should reveal only half of its arguments without waiting to see what other agents are going to do, while another strategy might specify that an agent should wait and see what arguments are revealed by others, before deciding how to respond. We can therefore define an argumentation mechanism.

Definition 5.19 (Argumentation mechanism) *Given an argumentation framework $AF = \langle \mathcal{A}, \mathcal{R} \rangle$ and semantics $\mathcal{S}$, an* argumentation mechanism *is defined as $\mathcal{M}^{\mathcal{S}}_{AF} = (\Sigma_1, \ldots, \Sigma_I, g(\cdot))$, where Σ_i is an argumentation strategy space of agent i*

MD Concept	ArgMD Instantiation
Agent type $\theta_i \in \Theta_i$	Agent's arguments $\theta_i = \mathcal{A}_i \subseteq \mathcal{A}$
Outcome $o \in \mathcal{O}$	Accepted arguments $Acc(.) \subseteq \mathcal{A}$
Utility $u_i(o, \theta_i)$	Preferences over $2^{\mathcal{A}}$ (what arguments end up being accepted)
Social choice function $f : \Theta_1 \times \ldots \times \Theta_I \to \mathcal{O}$	$f(\mathcal{A}_1, \ldots, \mathcal{A}_I) = Acc(\langle A_1 \cup \ldots \cup \mathcal{A}_I, \mathcal{R} \rangle, \mathcal{S})$ by some argument acceptability criterion
Mechanism $\mathcal{M} = (\Sigma, g(\cdot))$ where $\Sigma = \Sigma_1 \times \cdots \times \Sigma_I$ and $g : \Sigma \to \mathcal{O}$	Σ_i is an argumentation strategy, $g : \Sigma \to 2^{\mathcal{A}}$
Direct mechanism: $\Sigma_i = \Theta_i$	$\Sigma_i = 2^{\mathcal{A}}$ (every agent reveals a set of arguments)
Truth revelation	Revealing $\mathcal{A}_i$

Table 5.3: Abstract argumentation as a mechanism.

and $g : \Sigma_1 \times \ldots \times \Sigma_I \to 2^{\mathcal{A}}$.

Note that in the above definition, the notion of dialogue strategy is broadly construed and would depend on the protocol used. In a *direct* mechanism, however, the strategy spaces of the agents are restricted so that they can only reveal a subset of arguments. Due to the revelation principle, this will be sufficient for the analysis in the rest of the chapter.

Definition 5.20 (Direct argumentation mechanism) *Given an argumentation framework* $AF = \langle \mathcal{A}, \mathcal{R} \rangle$ *and semantics* $\mathcal{S}$*, a* direct argumentation mechanism *is defined as* $\mathcal{M}^{\mathcal{S}}_{AF} = (\Sigma_1, \ldots, \Sigma_I, g(\cdot))$*, where* $\Sigma_i = 2^{\mathcal{A}}$ *and* $g : \Sigma_1 \times \ldots \Sigma_I \to 2^{\mathcal{A}}$.

Table 5.3 summarizes the mapping of multiagent abstract argumentation as an instance of a mechanism design problem [42].

5.4 Case Study: Implementing the Grounded Semantics

In this section, I demonstrate the power of our ArgMD approach by showing how it can be used to systematically analyze the strategic incentives imposed by a well-established argument evaluation criterion [39, 43]. In particular, I specify a direct-revelation argumentation mechanism, in which agents' strategies are to reveal sets of arguments, and where the mechanism calculates the outcome using skeptical (grounded) semantics.[4] I show that, in general, this mechanism gives rise to strategic manipulation. However, under some conditions, this mechanism turns out to be strategyproof.

In a direct argumentation mechanism, each agent *i*'s available actions are $\Sigma_i = 2^{\mathcal{A}}$. We will refer to a specific action (i.e., set of declared arguments) as $\mathcal{A}^{\circ}_i \in \Sigma_i$. The following mechanism calculates the grounded extension given the union of all arguments revealed by agents.

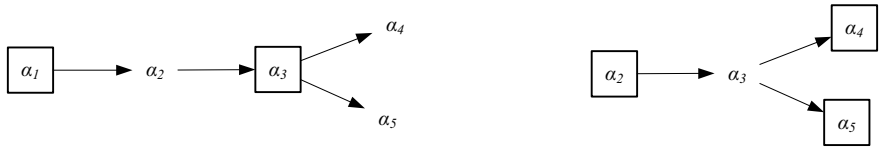
[4] In the remainder of the chapter, I will use the term *skeptical* to refer to *skeptical grounded*.

Definition 5.21 (Grounded direct argumentation mechanism) *A* grounded direct argumentation mechanism *for argumentation framework* $\langle \mathcal{A}, \mathcal{R} \rangle$ *is* $\mathcal{M}_{AF}^{grnd} = (\Sigma_1, \ldots, \Sigma_I, g(.))$ *where:*

- $\Sigma_i \in 2^{\mathcal{A}}$ *is the set of strategies available to each agent;*
- $g : \Sigma_1 \times \cdots \times \Sigma_I \rightarrow 2^{\mathcal{A}}$ *is an outcome rule defined as:* $g(\mathcal{A}_1^\circ, \ldots, \mathcal{A}_I^\circ) = Acc(\langle \mathcal{A}_1^\circ \cup \cdots \cup \mathcal{A}_I^\circ, \mathcal{R} \rangle, \mathcal{S}^{grnd})$ *where* $\mathcal{S}^{grnd}$ *denotes skeptical grounded acceptability semantics.*

To simplify our analysis, let us assume that agents can only lie by hiding arguments, and not by making up arguments. Formally, this means that $\forall i, \Sigma_i \in 2^{\mathcal{A}_i}$. For the sake of illustration, consider a particular family of preferences that agents may have. According to these preferences, every agent attempts to maximize the number of arguments in $\mathcal{A}_i$ that end up being accepted. We call this preference criteria the *individual acceptability maximizing preference*.

Definition 5.22 (Acceptability maximizing preferences) *An agent i has* individual acceptability maximizing preferences *if and only if* $\forall o_1, o_2 \in \mathcal{O}$ *such that* $|o_1 \cap \mathcal{A}_i| \geq |o_2 \cap \mathcal{A}_i|$*, we have* $u_i(o_1, \mathcal{A}_i) \geq u_i(o_2, \mathcal{A}_i)$.



(a) Argument graph in case of full revelation

(b) Argument graph with α_1 withheld

Figure 5.4: Hiding an argument is beneficial (case of acceptability maximizers).

The following example explores incentives with mechanism $\mathcal{M}_{AF}^{grnd}$.

Example 5.10 *Consider grounded direct argumentation mechanism with three agents x, y, and z with types* $\mathcal{A}_x = \{\alpha_1, \alpha_4, \alpha_5\}$, $\mathcal{A}_y = \{\alpha_2\}$, *and* $\mathcal{A}_z = \{\alpha_3\}$, *respectively. And suppose that the defeat relation is defined as follows:* $\mathcal{R} = \{(\alpha_1, \alpha_2), (\alpha_2, \alpha_3), (\alpha_3, \alpha_4), (\alpha_3, \alpha_5)\}$. *If each agent reveals its true type (i.e.,* $\mathcal{A}_x^\circ = \mathcal{A}_x$*;* $\mathcal{A}_y^\circ = \mathcal{A}_y$*; and* $\mathcal{A}_z^\circ = \mathcal{A}_z$*), then we get the argument graph depicted in Figure 5.4(a). The mechanism outcome rule produces the outcome* $o = \{\alpha_1, \alpha_3\}$. *If agents have individual acceptability maximizing preferences, with utilities equal to the number of arguments accepted, then:* $u_x(o, \{\alpha_1, \alpha_4, \alpha_5\}) = 1$*;* $u_y(o, \{\alpha_3\}) = 1$*; and* $u_z(o, \{\alpha_2\}) = 0$.

It turns out that the mechanism is susceptible to strategic manipulation, even if we suppose that agents do not lie by making up arguments (i.e., they may only withhold some arguments). In this case, for both agents y and z, revealing their true types weakly dominates revealing nothing at all. However, it turns out that agent x is better off revealing $\{\alpha_4, \alpha_5\}$. By withholding α_1, the resulting argument network becomes as depicted in Figure 5.4(b), for which the output rule produces the outcome $o' = \{\alpha_2, \alpha_4, \alpha_5\}$. This outcome yields utility 2 to agent x, which is better than the truth-revealing strategy. Thus, given an arbitrary argumentation framework AF and agents with acceptability maximizing preferences, mechanism $\mathcal{M}_{AF}^{grnd}$ is *not* strategyproof.

The following theorem provides a full characterization of strategyproof mechanisms for skeptical argumentation frameworks for agents with acceptability maximizing preferences.

Theorem 5.3 *Let AF be an arbitrary argumentation framework, and let $\mathcal{E}_{\mathcal{GR}}(AF)$ denote its grounded extension. Mechanism $\mathcal{M}_{AF}^{grnd}$ is strategyproof for agents with acceptability maximizing preferences if and only if AF satisfies the following condition: $\forall i \in I, \forall S \subseteq \mathcal{A}_i$ and $\forall \mathcal{A}_{-i}$, we have $|\mathcal{A}_i \cap \mathcal{E}_{\mathcal{GR}}(\langle \mathcal{A}_i \cup \mathcal{A}_{-i}, \mathcal{R} \rangle)| \geq |\mathcal{A}_i \cap \mathcal{E}_{\mathcal{GR}}(\langle (\mathcal{A}_i \backslash S) \cup \mathcal{A}_{-i}, \mathcal{R} \rangle)|$.*

Although the theorem gives us a full characterization, it is difficult to apply in practice. In particular, the theorem does not give us an indication of how agents (or the mechanism designer) can identify whether the mechanism is strategyproof for a class of argumentation frameworks by appealing to their graph-theoretic properties. Below, we provide an intuitive, graph-theoretic condition that is sufficient to ensure that $\mathcal{M}_{AF}^{grnd}$ is strategyproof when agents have focal arguments.

Let $\alpha, \beta \in \mathcal{A}$. We say that α *indirectly defeats* β, written $\alpha \hookrightarrow \beta$, if and only if there is an odd-length path from α to β in the argument graph.

Theorem 5.4 *Suppose agents have individual acceptability maximizing preferences. If each agent's type corresponds to a conflict-free set of arguments that does not include (in)direct defeats (formally $\forall i \nexists \alpha_1, \alpha_2 \in \mathcal{A}_i$ such that $\alpha_1 \hookrightarrow \alpha_2$), then $\mathcal{M}_{AF}^{grnd}$ is strategyproof.*

Note that $\hookrightarrow$ is over all arguments in $\mathcal{A}$. Intuitively, the condition in the theorem states that *all* arguments of every agent must be conflict-free (i.e., consistent), both explicitly and implicitly. Explicit consistency implies that no argument defeats another. Implicit consistency implies that other agents cannot possibly present a set of arguments that reveal an indirect defeat among one's own arguments. More concretely, in Example 5.10 and Figure 5.4, while agent x's argument set $\mathcal{A}_x = \{\alpha_1, \alpha_4, \alpha_5\}$ is conflict-free, when agents y and z presented their own ar-

guments α_2 and α_3, they revealed an implicit conflict in x's arguments. In other words, they showed that x contradicts itself (i.e., committed a *fallacy*).

In addition to characterizing a sufficient graph-theoretic condition for strategyproofness, Theorem 5.4 is useful for individual agents. As long as the agent knows that it is *not* possible for a path to be created that causes an (in)direct defeat among its arguments (i.e., a fallacy to be revealed), then the agent is best off revealing all its arguments. The agent only needs to know that no argument imaginable can reveal conflicts among its own arguments.

Is the *sufficient* condition in Theorem 5.4 also *necessary* for revealing all arguments truthfully? Example 5.11 shows that this is not the case. In particular, for certain argumentation frameworks, an agent may have truth-telling as a dominant strategy despite the presence of indirect defeats among its own arguments.

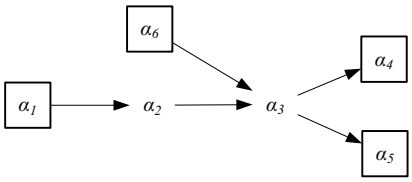


Figure 5.5: Argument graph (see Example 5.11).

Example 5.11 *Consider the variant of Example 5.10 with the additional argument* α_6 *and defeat* (α_6, α_3). *Let the agent types be* $\mathcal{A}_x = \{\alpha_1, \alpha_4, \alpha_5, \alpha_6\}$, $\mathcal{A}_y = \{\alpha_2\}$, *and* $\mathcal{A}_z = \{\alpha_3\}$, *respectively. The full argument graph is depicted in Figure 5.5. Under full revelation, the mechanism outcome rule produces the outcome* $o = \{\alpha_1, \alpha_4, \alpha_5, \alpha_6\}$.

Note that in Example 5.11, truth revelation is now a dominant strategy for x (since it gets all its arguments accepted) despite the fact that $\alpha_1 \hookrightarrow \alpha_4$ and $\alpha_1 \hookrightarrow \alpha_5$. This hinges on the presence of an argument (namely α_5) that cancels out the negative effect of the (in)direct self-defeat among x's own arguments.

6 The Argument Interchange Format

To facilitate argumentation among agents in an open system, it is essential to have a common language for argument representation. One community-led effort toward this is the *Argumentation Interchange Format (AIF)*. Here, I give a very brief overview of the AIF, and point the reader elsewhere for further details [11].

The core AIF has two types of nodes: *information nodes* (or *I-nodes*) and *scheme nodes* (or *S-nodes*). These are represented by two disjoint sets, $\mathcal{N}_I \subset \mathcal{N}$ and $\mathcal{N}_S \subset \mathcal{N}$, respectively. Information nodes are used to represent *passive* information contained in an argument, such as a claim, premise, data, etc. S-nodes capture the application of *schemes* (i.e., patterns of reasoning). Such schemes may be domain-independent patterns of reasoning, which resemble rules of inference in deductive logics but broadened to include non-deductive inference. The schemes themselves belong to a class, $\mathcal{S}$, and are classified into the types: *rule of inference scheme*, *conflict scheme*, and *preference scheme*. We denote these using the disjoint sets $\mathcal{S}^R$, $\mathcal{S}^C$, and $\mathcal{S}^P$, respectively. The predicate $(\mathsf{uses} : \mathcal{N}_S \times \mathcal{S})$ is used to express the fact that a particular scheme node uses (or instantiates) a particular scheme. The AIF thus provides an ontology for expressing schemes and instances of schemes, and constrains the latter to the domain of the former via the function uses, i.e., $\forall n \in \mathcal{N}_S, \exists s \in \mathcal{S}$ such that $\mathsf{uses}(n,s)$.

The present ontology has three different types of scheme nodes: *rule of inference application nodes* (or *RA-nodes*), *preference application nodes* (or *PA-nodes*) and *conflict application nodes* (or *CA-nodes*). These are represented as three disjoint sets: $\mathcal{N}_S^{RA} \subseteq \mathcal{N}_S$, $\mathcal{N}_S^{RA} \subseteq \mathcal{N}_S$, and $\mathcal{N}_S^{CA} \subseteq \mathcal{N}_S$, respectively. The word "application" on each of these types was introduced in the AIF as a reminder that these nodes function as instances, not classes, of possibly generic inference rules. Intuitively, $\mathcal{N}_S^{RA}$ captures nodes that represent (possibly non-deductive) rules of inference, $\mathcal{N}_S^{CA}$ captures applications of criteria (declarative specifications) defining conflict (e.g., among a proposition and its negation, etc.), and $\mathcal{N}_S^{RA}$ are applications of (possibly abstract) criteria of preference among evaluated nodes.

The AIF specification does not type its edges. The (informal) semantics of edges can be inferred from the types of nodes they connect. One of the restrictions is that no outgoing edge from an I-node can be directed directly to another I-node. This ensures that the type of any relationship between two pieces of information must be specified explicitly via an intermediate S-node.

Definition 5.23 ***(Argument network)*** *An* argument network Φ *is a graph with: (i) a set* $\mathcal{N} = \mathcal{N}_I \cup \mathcal{N}_S$ *of vertices (or* nodes*); and (ii) a binary relation* $\xrightarrow{edge}$: $\mathcal{N} \times \mathcal{N}$ *representing edges, where* $\nexists (i,j) \in \xrightarrow{edge}$ *where both* $i \in \mathcal{N}_I$ *and* $j \in \mathcal{N}_I$.

A *simple* argument can be represented by linking a set of premises to a conclusion.

Definition 5.24 ***(Simple argument)*** *A* simple argument*, in network* Φ *and schemes* $\mathcal{S}$*, is a tuple* $\langle P, \tau, c \rangle$ *where: (i)* $P \subseteq \mathcal{N}_I$ *is a set of nodes denoting premises; (ii)* $\tau \in \mathcal{N}_S^{RA}$ *is a rule of inference application node; (iii)* $c \in \mathcal{N}_I$ *is a node denoting the conclusion, such that* $\tau \xrightarrow{edge} c$, $\mathsf{uses}(\tau, s)$ *where* $s \in \mathcal{S}$*, and* $\forall p \in P$ *we have* $p \xrightarrow{edge} \tau$.

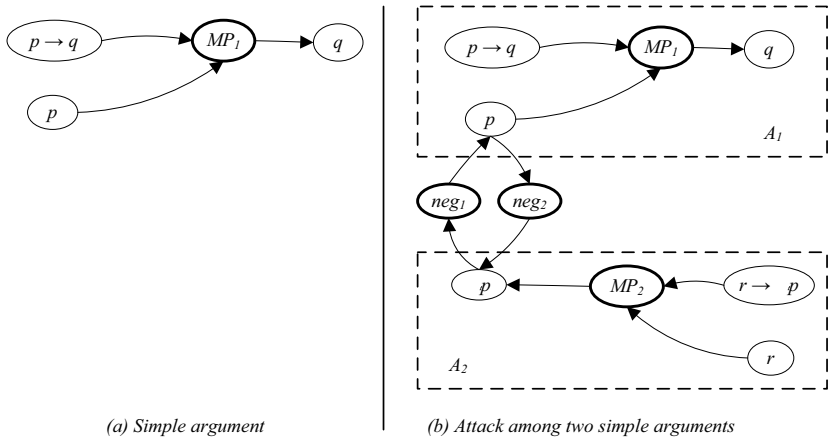


Figure 5.6: Examples of simple arguments. S-nodes denoted with a thicker border.

Following is a description of a simple argument in propositional logic, depicted in Figure 5.6(a).

Example 5.12 ***(Simple argument)*** *The tuple $A_1 = \langle\{p, p \rightarrow q\}, MP_1, q\rangle$ is a simple argument in propositional language $\mathcal{L}$, where p, $(p \rightarrow q) \in \mathcal{N}_I$ are nodes representing premises, and $q \in \mathcal{N}_I$ is a node representing the conclusion. In between them, the node $MP_1 \in \mathcal{N}_S^{RA}$ is a rule of inference application node (i.e., RA-node) that uses the* modus ponens *natural deduction scheme, which can be formally written as follows:* $\mathsf{uses}(MP_1, \forall A, B \in \mathcal{L}\ \frac{A \qquad A \rightarrow B}{B})$.

An attack or conflict from one information or scheme node to another information or scheme node is captured through a CA-node, which captures the type of conflict. The attacker is linked to the CA-node, and the CA-node is subsequently linked to the attacked node. Note that since edges are directed, each CA-node captures attack in one direction. Symmetric attack would require two CA-nodes, one in each direction. The following example describes a conflict between two simple arguments (see Figure 5.6(b)).

Example 5.13 ***(Simple arguments in conflict)*** *Recall the simple argument $A_1 = \langle\{p, p \rightarrow q\}, MP_1, q\rangle$. And consider another simple argument $A_2 = \langle\{r, r \rightarrow \neg p\}, MP_2, \neg p\rangle$. Argument A_2 undermines A_1 by supporting the negation of the latter's premise. This (symmetric) propositional conflict is captured through two CA-nodes: neg_1 and neg_2, both of which instantiate a conflict scheme based on propositional contraries.*

Note that the AIF language is less abstract than Dung's abstract argument graphs, but is more abstract than approaches that describe the internal contents of

logical arguments. Moreover, the AIF was deliberately given only semi-formal semantics, allowing for it to be adapted according to one's need, for example with a particular language for describing the internal contents of information nodes, or by committing to edges with specific formal semantics. It has been shown that the AIF can be adapted for creating ontologies using Semantic Web standards for annotating natural language arguments using Walton-style schemes (see for example [47]). Having said that, the AIF is still a young effort in need of further refinement and proof-of-concept applications.

7 Conclusion

I gave an overview of the emerging field of argumentation in multiagent systems. I introduced some basic definitions of the argument and relationships between arguments. I then gave an overview of protocols that govern argumentation dialogues among agents, before moving to the issue of strategic argumentation. I closed the discussion with a brief overview of efforts toward a common ontology for enabling the exchange of arguments.

This field of study is still in its infancy. While much is understood about the properties of argumentation semantics [2] and the termination and complexity properties of dialogue protocols [13], strategic aspects are still underexplored. Game-theoretic tools have proven indispensable to the understanding of other forms of interaction in multiagent systems, such as auction and voting protocols, as illustrated elsewhere in this book. Yet the connection between argumentation processes and game theory still has a long way to go.

Another important challenge is understanding how computational models of argument relate to how people actually argue. This is crucial to the task of programming agents capable of arguing with, and successfully persuading people [26]. The models explored in this chapter mostly overlook questions of psychological plausibility [40]. There is an opportunity for cross-fertilization between computational argumentation and human reasoning research [50].

Acknowledgment

Some of the contents of this chapter are based on excerpts from my previous work. I would like to thank all my coauthors on those articles, especially Kate Larson and Chris Reed. I would also like to apologize for the omission of much important work, which I simply did not have space for due to a limited number of pages.